UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843

Special Topic in Data Engineering

Tutorial 3: AI Algorithm

LECTURER: DR. ARYATI BINTI BAKRI

NO.	NAME	MATRIC NUMBER
1.	IMAN ABADI BINN MOHD NIZWAN	A23CS0084
2.	MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

Table of Contents

1.0 Understanding of Image Classification

Image classification is the process of categorizing images into predefined classes. In machine learning, models are trained to recognize visual patterns and assign the correct label to an image (IBM, n.d.). It falls under the field of computer vision, which uses machine learning and neural networks to enable computers to interpret visual data such as images and videos. AI systems require structured training through labels or learned statistical patterns to classify an object compared to humans that can classify objects instinctively.

1.1 Types of Image Classification

Rule-based classification relies on manually defined rules and feature selection by domain experts. Techniques include template matching, which compares regions of an image against a reference template and thresholding, which converts pixel intensities to binary values to separate features from the background. It offers interpretability but requires significant manual effort (IBM, 2025). The diagram for rule-based classification is shown in Figure 1.

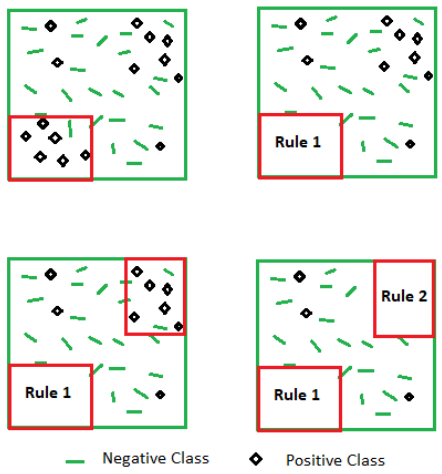

Figure 1: Rule-Based Classification Diagram (GeekforGeeks, n.d.)

Statistical classification automatically learns patterns from large labelled datasets where convolutional neural networks (CNNs) are most commonly used due to its powerful architecture. CNNs use three layer types, each increasing in complexity to identify parts of the image shown in Figure 2. As the data moves through the various CNN layers, an increased number of components become recognized until the image can be classified. The general workflow involves data collection and preprocessing, model selection, training and validation, feature extraction and finally evaluation and deployment (IBM, 2025).

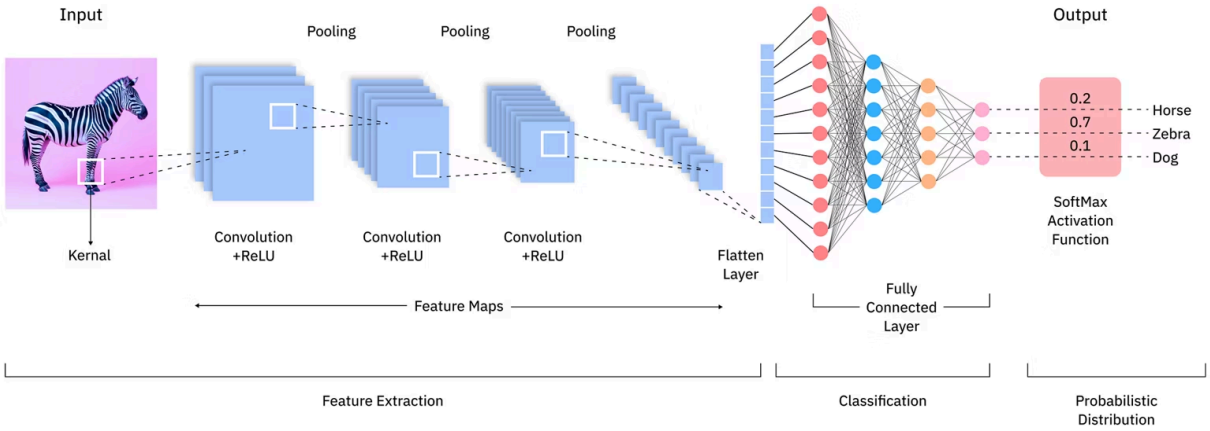

Figure 2: CNN Architecture Diagram (IBM, n.d.)

1.2 Common Models and Algorithms

Various models and algorithms have been developed for image classification. They range from approaches like K-nearest neighbors (KNN), random forests, and support vector machines (SVM), to architectures such as AlexNet, GoogLeNet and ResNet. Each method offers different strengths in terms of accuracy, scalability and complexity (IBM, 2025). Popular deep learning architectures include:

- AlexNet: A fundamental CNN that uses ReLU activation functions instead of sigmoid, contributing to faster and more effective training.
- GoogLeNet/Inception: Developed by Google, this architecture employs parallel inception modules with different filter sizes, whose outputs are combined for a single result.
- ResNet: Introduces residual (shortcut) connections that allow data to bypass certain layers, enabling the training of networks up to 152 layers deep.

2.0 Understanding of CNN

Convolutional neural networks use three-dimensional data for image classification and object recognition tasks (IBM, 2021). They are a specialized type of deep learning architecture designed to process grid-like data, such as images through a sequence of learned transformations.

Before CNNs, feature extraction was done manually to identify objects in images which was time consuming. However, convolutional neural networks now provide a more scalable approach to image classification and object recognition tasks, leveraging principles from linear algebra, specifically matrix multiplication, to identify patterns within an image (IBM, 2021).

2.1 Architecture of a CNN

A convolutional neural network has three main types of layers namely the convolutional layer, the pooling layer and the fully-connected (FC) layer (IBM, 2021). The layers can be seen in Table 1.

With each layer, the CNN increases in its complexity, identifying greater portions of the image. The initial layers focus on simple features, such as colors and edges. As the image data progresses through the layers of the CNN, it starts to recognize larger elements or shapes of the object until the intended object is finally identified (IBM, 2021).

Table 1: CNN Architectural Layers

Layer	Description
Convolution (Conv2D)	Slides small filters over the image to make feature maps that show patterns like edges, corners, and textures.
ReLU	Turns any negative number into 0. This lets the network learn more than simple straight-line patterns.
Pooling (MaxPooling2D)	Shrinks the feature maps by keeping only the strongest values. This makes the model faster and helps it still work if the object moves a little.
Flatten	Changes the 2D feature maps into one long list of numbers so the dense layers can use them.
Dense (fully connected)	Mix the features together to make the final choice.
Softmax (output)	Gives a score from 0 to 1 for each of the 10 classes, and all scores add up to 1.

2.2 Applications of CNNs

Convolutional neural networks power image recognition and computer vision tasks across multiple domains. In marketing, social media platforms use CNNs to suggest who might be in a photograph. In healthcare, computer vision has been incorporated into radiology technology, enabling doctors to better identify cancerous tumors. In retail, visual search allows brands to recommend items complementing an existing wardrobe. In the automotive industry, CNN-based features such as lane line detection are improving driver and passenger safety (IBM, 2021).

3.0 Evaluation of CNN

Rather than relying solely on accuracy, the model used multiple complementing metrics to measure unknown test data after training.

Metric	**Meaning**
Accuracy	Fraction of correctly classified images. CIFAR-10 classes are balanced, so accuracy is meaningful.
Loss	How far predictions are from the true labels. Lower is better; comparing train vs. validation loss reveals overfitting.
Precision	Of all images predicted as class X, how many were actually X.
Recall	Of all true class-X images, how many the model found.
F1-score	Harmonic mean of precision and recall. A single balanced score.
Confusion matrix	Table of true vs. predicted classes that shows which classes get confused with each other.

If training accuracy is high but validation/test accuracy is much lower, the model overfits. Use the classification report.

4.0 Steps Hands-on in Python

The implementation uses TensorFlow or Keras. The steps below load CIFAR-10, normalise it, train a baseline ANN, then train the baseline CNN.

4.1 Imports and loading the datasets

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

```
(X_train, y_train), (X_test,y_test) = datasets.cifar10.load_data()
X_train.shape
```

```
(50000, 32, 32, 3)
```

```
X_test.shape
```

```
(10000, 32, 32, 3)
```

Figure 4.1 Imports and loading the datasets codes

There are 50,000 training images and 10,000 test images, each 32×32 with 3 (RGB) channels.

4.2 Reshape labels and define classes

The labels come as a 2-D array of shape (50000, 1). A 1-D array is more convenient, so it need to be reshaped.

```
y_train = y_train.reshape(-1,)
y_train[:5]
```

```
array([6, 9, 9, 4, 1], dtype=uint8)
```

```
y_test = y_test.reshape(-1,)
```

```
classes = ["airplane","automobile","bird","cat","deer","dog","frog","horse","ship","truck"]
```

Figure 4.2 Reshape labels and define classes codes

Figure 4.3 Sample CIFAR-10 images with class labels

4.3 Normalization

Each pixel value ranges from 0 to 255. Dividing by 255 scales the data into the 0–1 range, which helps the network train faster and more stably.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 4.4 Normalization codes

4.4 Baseline ANN

First, a plain ANN is built for comparison. After 5 rounds (epochs) it only gets about 48 to 49% accuracy. This is weak for images because the ANN does not care about where the pixels are in the image.

```
ann = models.Sequential([
        layers.Flatten(input_shape=(32,32,3)),
        layers.Dense(3000, activation='relu'),
        layers.Dense(1000, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])

ann.compile(optimizer='SGD',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

Figure 4.5 Baseline ANN code

```
Classification Report:
              precision    recall  f1-score   support

           0       0.58      0.52      0.55      1000
           1       0.69      0.53      0.60      1000
           2       0.37      0.40      0.38      1000
           3       0.30      0.52      0.38      1000
           4       0.48      0.34      0.40      1000
           5       0.44      0.32      0.37      1000
           6       0.62      0.38      0.47      1000
           7       0.56      0.54      0.55      1000
           8       0.57      0.69      0.63      1000
           9       0.51      0.66      0.57      1000

    accuracy                           0.49     10000
   macro avg       0.51      0.49      0.49     10000
weighted avg       0.51      0.49      0.49     10000
```

Figure 4.6 ANN classification report

4.5 Baseline CNN

The baseline CNN uses two convolutions with max-pooling blocks, then a dense head. Trained for 10 epochs with the Adam optimizer, it reaches about 70% test accuracy which is a large jump over the ANN, with far less computation thanks to pooling.

```python
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```python
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

```python
cnn.fit(X_train, y_train, epochs=10)
```

Figure 4.7 Baseline CNN code

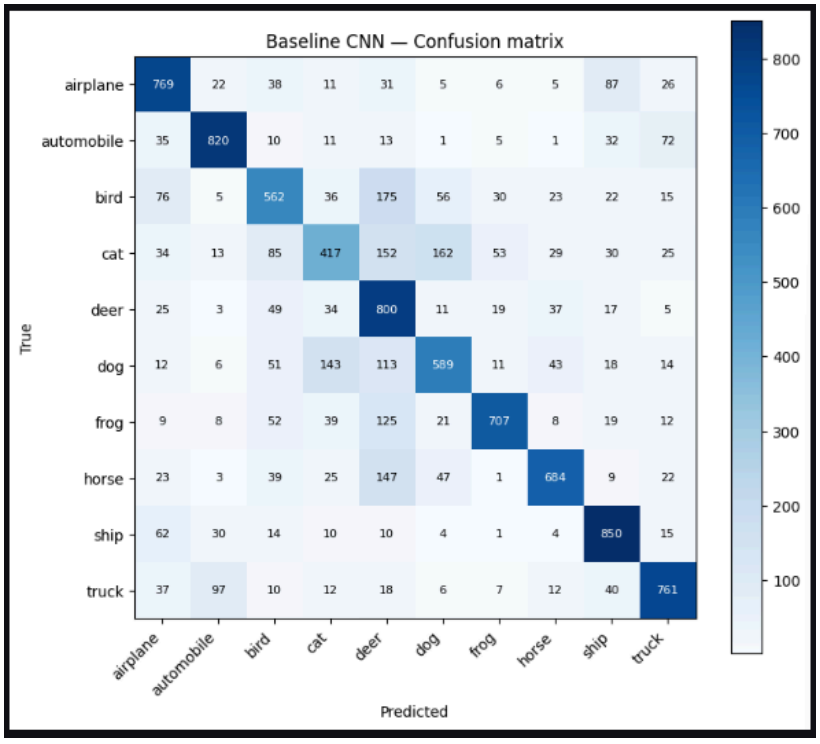

Figure 4.8 Confusion Matrix for Base CNN

Baseline CNN — Classification Report:

	precision	recall	f1-score	support
airplane	0.71	0.77	0.74	1000
automobile	0.81	0.82	0.82	1000
bird	0.62	0.56	0.59	1000
cat	0.57	0.42	0.48	1000
deer	0.51	0.80	0.62	1000
dog	0.65	0.59	0.62	1000
frog	0.84	0.71	0.77	1000
horse	0.81	0.68	0.74	1000
ship	0.76	0.85	0.80	1000
truck	0.79	0.76	0.77	1000
accuracy			0.70	10000
macro avg	0.71	0.70	0.69	10000
weighted avg	0.71	0.70	0.69	10000

Figure 4.9 CNN classification report

By round 10 the first CNN got about 80% on the training images but only about 70% on the test images, while the ANN got about 48%. This shows that CNN is much better for images. The per-class table below shows the first CNN already has trouble with cat, bird and dog.

5.0 Weakness of CNN Model

The first CNN gets about 70% accuracy, but it has some clear weak points:

1. Overfitting. It gets about 80% on the training images but only about 70% on the test images. This gap of about 10 percent means it is starting to memorise the training images.
2. No anti-overfitting tools. It has no Dropout or Batch Normalization to stop overfitting or make training steadier.
3. No data augmentation. It only sees the original images, so it does not handle flipped, shifted, or turned images well.
4. Too simple. It has only two convolution layers with few filters, so it cannot learn many patterns.
5. Mixes up similar classes. The lowest scores are on cat (F1 0.48), bird (0.59) and dog (0.62). These animals look alike, so the model confuses them.
6. Fixed training setup. The learning rate never changes and there is no early stopping, so the model either trains too little or wastes time after it stops getting better.

6.0 Enhancement of CNN Model

To fix those weak points, a better model called new_cnn is built. It adds:

- Data augmentation. This makes the training images more varied, so the model overfits less.
- Batch Normalization after the convolution layers. This makes training faster and steadier.
- Dropout. This turns off some neurons at random during training to stop overfitting.
- More layers and more filters with padding so the model can learn more patterns.
- EarlyStopping and ReduceLROnPlateau. These stop training when it stops getting better, and lower the learning rate by themselves.

6.1 Building the enhanced model

```
new_cnn = models.Sequential([
    layers.Input(shape=(32, 32, 3)),

    # Data augmentation (only active during training)
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),

    # Block 1
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Block 2
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),

    # Block 3
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.4),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

new_cnn.compile(optimizer='adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])

new_cnn.summary()
```

Figure 6.1 Enhanced CNN code

The new model has 552,874 parameters in total (551,722 that it learns). It is bigger than the first CNN and can learn more, but the extra tools keep it from overfitting.

6.2 Training with callbacks

The test set is used to watch the model during training. The callbacks stop training when it stops getting better, so 20 rounds is just the most it will train, not always the full amount.

```
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=4,
                                     restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                         patience=2, min_lr=1e-5),
]

history_new = new_cnn.fit(X_train, y_train,
                          epochs=20,
                          validation_data=(X_test, y_test),
                          callbacks=callbacks)
```

Figure 6.2 Training with callbacks for enhanced CNN

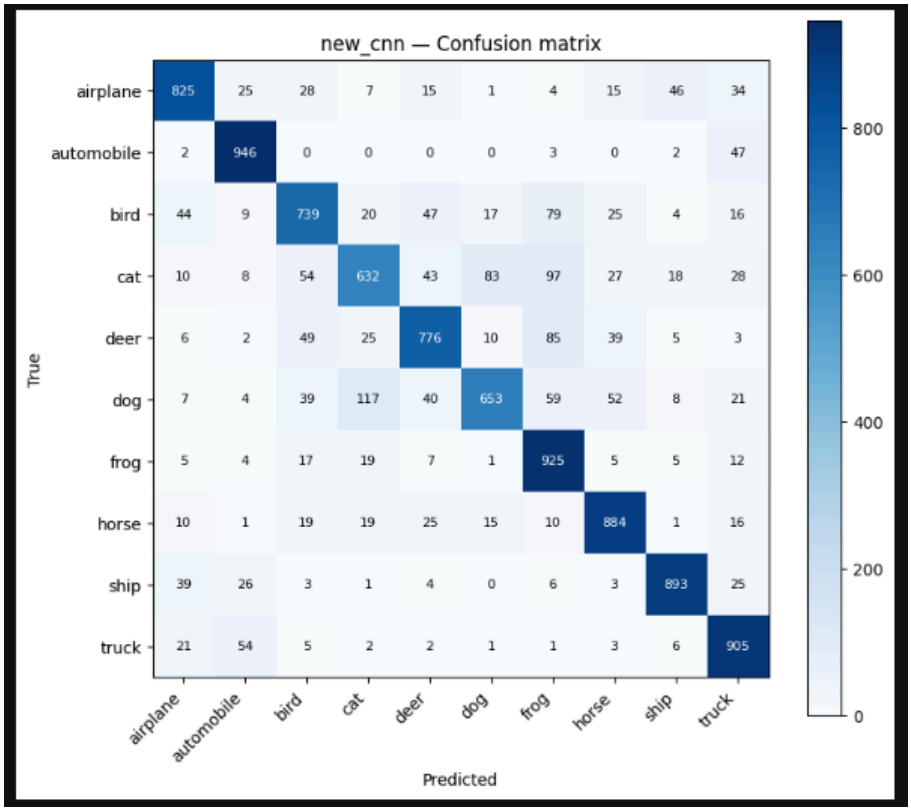

Figure 6.3 Confusion Matrix for enhanced CNN

```
new_cnn — Classification Report:
              precision    recall  f1-score   support

    airplane       0.85      0.82      0.84      1000
  automobile       0.88      0.95      0.91      1000
        bird       0.78      0.74      0.76      1000
         cat       0.75      0.63      0.69      1000
        deer       0.81      0.78      0.79      1000
         dog       0.84      0.65      0.73      1000
        frog       0.73      0.93      0.82      1000
       horse       0.84      0.88      0.86      1000
        ship       0.90      0.89      0.90      1000
       truck       0.82      0.91      0.86      1000

    accuracy                           0.82     10000
   macro avg       0.82      0.82      0.82     10000
weighted avg       0.82      0.82      0.82     10000
```

Figure 6.4 Enhanced CNN classification report

In this run the model kept getting better, so it trained for all 20 rounds. The learning rate was lowered automatically (from 0.001 to 0.0005 to 0.00025) when progress slowed down. The best round reached about 82% accuracy with a loss of about 0.53 on the test set, which is much better than the first CNN

7.0 Comparison (CNN vs New CNN)

Both models are evaluated on the same 10,000-image test set.

7.1 Side-by-side comparison

Measure	**First CNN**	**new_cnn**
Test accuracy	0.70	0.82
Test loss	about 0.9	0.53 (lower is better)
Precision (average)	0.71	0.82
Recall (average)	0.70	0.82
F1-score (average)	0.69	0.82
Training accuracy	about 0.80	about 0.77 (no overfit)
Anti-overfitting tools	None	BatchNorm, Dropout, Augmentation

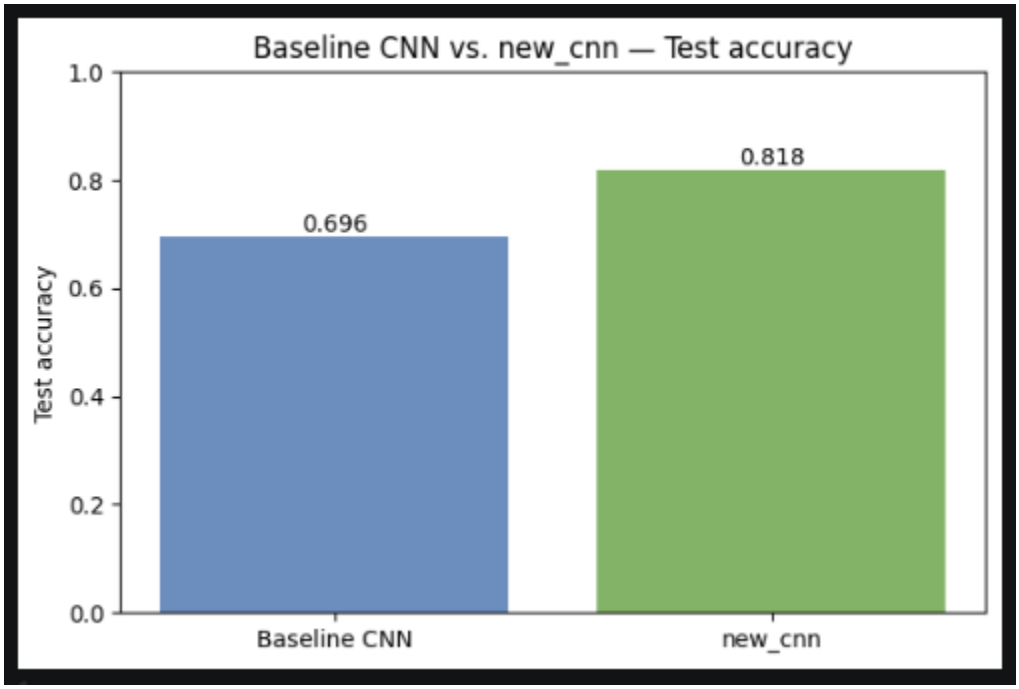

Figure 7.1 Test Accuracy Bar Chart

8.0 Results & Discussion

The first CNN got about 70% on the test set, but it overfit. It reached about 80% on the training images while the test stayed near 70%, and the higher test loss shows the same gap. The new_cnn on the other hand achieved about 82% on the test set (loss about 0.53). Its training accuracy (about 77%) is actually lower than its test accuracy. The random changes to the images make training harder, so the model learns the general idea and stops memorizing. The model is 12 percent better not only in accuracy but also in precision, recall and F1-score (average went from 0.69 to 0.82). So it is better for every class, not just on average. Every class improved. The hardest ones are still the animals that look alike. The cat went from 0.48 to 0.69, bird from 0.59 to 0.76, and dog from 0.62 to 0.73 (F1-score), but all of them showed significant improvements of over then 10 points.

The improvements are due to the addition of anti-overfitting practices and tools to help make a better generalization of the data rather than memorizing them. The use of data augmentation lets the model see flipped, turned and zoomed images, so it works better on new images. Batch normalization made training steadier and faster, so more layers could be used to improve performance. The use of dropout stopped the model from relying on any single neuron, so it overfit less. EarlyStopping and ReduceLROnPlateau kept the best version of the model and lowered the learning rate on their own.

Additional improvements can be made so that accuracy could be pushed past 90% by using transfer learning (a model already trained on a big dataset, like ResNet or VGG), training for more rounds on a GPU or using newer model designs. Also, CIFAR-10 image datasets are very small (around 32 by 32), which limits how high the accuracy can go.

9.0 Reflection

IMAN ABADI BIN MOHD NIZWAN

Through this tutorial, a better understanding of why CNNs outperform plain ANNs for image data was gained. A theoretical understanding of CNN, especially the convolutional layers, was gained with the addition of implementing the full pipeline on CIFAR-10 giving a much clearer picture of how each layer results into the final prediction. Seeing how CNN outperforms ANN using the accuracy as measures shows proof from the hands-on application further solidified my understanding on why CNN is better for image classification. However, the CNN model shows signs of overfitting which was a significant challenge to evaluating the effectiveness of the model on unseen data. This gave the opportunity to learn on how to further improve the CNN model with multiple anti-overfitting tools such as using data augmentation and batch normalization to prevent the model from memorizing the data rather than generalizing it as overfitting can cause lower performance on unseen data. Improvement opportunities may come from using other types of CNN models such as ResNet, a deep residual learning approach for image recognition to have better performance, aiming for accuracy of over 90%.

MOHAMED ALIF FATHI BIN ABDUL LATIF

This tutorial provided valuable practical insight into the critical role that comprehensive evaluation metrics play in deep learning. While measuring baseline accuracy offers a surface-level view of a model's performance, analyzing the confusion matrix, precision, recall and F1-scores shows how easily a model can struggle with visually similar classes, such as cats and dogs. The most important lesson was how to methodically identify and address overfitting. Adaptive callbacks such as ReduceLROnPlateau and EarlyStopping were used to show how dynamic hyperparameter adjustment maximizes model capability while avoiding computational waste. To push the accuracy past 90%, transfer learning with pre-trained models like ResNet could be used, train for more rounds on a GPU or use newer model designs. However, because CIFAR-10 images are so small (only 32x32 pixels), the lack of detail limits how high the accuracy can go. In the future, this model should be tested on a different, real-world dataset with larger, high-quality images to get better results.

10.0 References

1. IBM. “Image Classification.” Ibm.com, 30 Sept. 2025, www.ibm.com/think/topics/image-classification
2. IBM. “Convolutional Neural Networks.” Ibm.com, 6 Oct. 2021, www.ibm.com/think/topics/convolutional-neural-networks
3. GeeksforGeeks. (2022, January 12). RuleBased Classifier Machine Learning. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/rule-based-classifier-machine-learning/